PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ FACULTAD DE CIENCIAS E INGENIERÍA INGENIERÍA INFORMÁTICA

Proyecto de Diseño y Desarrollo de Software
1INF54 Estándar del backend
Grupo 6F

Integrantes:

- Joseph Enrique Huaman Pérez 20185613
- Diego Bernardo Castilla Arnao Monteagudo 20196342
- Percy Leonardo Marca Rojas 20202682
- Lhía Hurtado Claros 20211729

Profesor: **Juan Carlos Fernández**

Lima, 18 de abril de 2025

Historial de Revisiones

Fecha	Versión	Descripción	Responsable
18/09/2025	1.0	Definición de estándares para desarrollo de software - Estándar BackEnd	Grupal 6F

Tabla de contenido

Historial de Revisiones 2
Tabla de contenido 3
1. Introducción 4
1.1. Propósito del Documento 4
1.2. Audiencia Objetivo 4
1.3. Alcance 4
2. Arquitectura 4
3. Estructura del Proyecto 5
4. Convenciones de Codificación 5
5. Manejo de Datos 6
6. API REST 6
7. Autenticación y Autorización 7
8. Manejo de Errores 7
9. Pruebas 8
10. Despliegue 8
11. Documentación 8
12. Seguridad 8
13. Rendimiento 9
14. Control de Versiones 9
15. Glosario 9

1. Introducción

1.1. Propósito del Documento

Definir las reglas técnicas, estructura, API y prácticas de desarrollo para el backend de Mora Pack: registro de pedidos, planificación/planificación con metaheurísticas (p. ej., Algoritmo Genético y una segunda técnica), y exposición de datos para el visualizador de mapa y paneles de desempeño.

1.2. Audiencia Objetivo

Equipo de desarrollo, QA, docentes evaluadores y cualquier colaborador que mantenga o despliegue el backend.

1.3. Alcance

Incluye arquitectura, paquetes, estándares de código, manejo de datos, API REST, seguridad, errores, pruebas, despliegue, documentación, rendimiento y gobernanza del código. No cubre el front/visualizador ni la simulación externa.

2. Arquitectura

La arquitectura backend seguirá un diseño de capas para asegurar modularidad y mantenibilidad:

- Capa de Presentación (API): Encargada de manejar las solicitudes HTTP, recibir datos de entrada (como pedidos o parámetros de simulación), interactuar con la capa de servicios y devolver respuestas adecuadas a los usuarios o al visualizador de operaciones.
- Capa de Lógica de Negocio (Servicios): Contiene las reglas de negocio del sistema MoraPack. Aquí se procesan los pedidos, se ejecutan los algoritmos de planificación (metaheurísticas), se gestionan los escenarios de simulación (día a día, semanal, colapso) y se coordinan las operaciones de monitoreo.
- Capa de Acceso a Datos (Repositorios): Responsable de interactuar con la base de datos para realizar operaciones CRUD sobre entidades como pedidos, vuelos, almacenes, corridas de algoritmos y métricas de desempeño.
- Capa de Dominio: Define las entidades principales del modelo de negocio, como Pedido, Cliente, Vuelo, Almacén o Corrida. Estas representan los objetos centrales del sistema y mantienen la coherencia de la información.

3. Estructura del Proyecto

Paquetes:

- controller: Para manejar las solicitudes HTTP (pedidos, planificador, monitoreo, experimentos).
- service: Para la lógica de negocio (ejecución de algoritmos, validación de escenarios, replanificación).
- repository: Para la interacción con la base de datos (pedidos, vuelos, corridas, métricas).
- model: Para las entidades de dominio (Pedido, Vuelo, Almacen, Corrida) y los DTOs usados en las APIs.
- util: Para clases de utilidad, como cargadores de escenarios, parsers de archivos y generadores de seeds.

Ejemplo de estructura:

```
src/
├── main/
│   ├── java/
│   │   └── com/
│   │       └── morapack/
│   │           └── backend/
│   │               ├── controller/
│   │               ├── service/
│   │               ├── repository/
│   │               ├── model/
│   │               ├── util/
│   │               └── application.properties
└── test/
```

4. Convenciones de Codificación

- Java: Seguir las buenas prácticas de programación en Java.
  - Nombres de clases en PascalCase.
  - Nombres de métodos y variables en camelCase.
  - Constantes en MAYÚSCULAS_CON_GUIONES.
  - Uso de comentarios Javadoc para clases y métodos públicos.
- Spring Boot:
  - Utilizar application.properties o application.yml para configurar parámetros como:
    Puerto del servidor.
    Conexión a la base de datos.
    Parámetros de seguridad.
    Configuraciones específicas de cada entorno (dev, test, prod).
- Organización de código:
  - Mantener un código limpio, modular y reutilizable.
  - Aplicar principios SOLID y separación de responsabilidades.

- Implementar pruebas unitarias para asegurar la calidad y reducir errores.
- Evitar duplicación de código mediante clases de utilidad y servicios bien definidos.

5. Manejo de Datos

- Base de Datos Relacional:
  Se usará MySQL para la persistencia de datos. El modelo de datos se definirá mediante JPA (Java Persistence API), con las entidades en la carpeta model.
- Repositorios:
  Se empleará Spring Data JPA para interactuar con la base de datos de manera eficiente, reduciendo el uso de código repetitivo para las operaciones CRUD.
- Gestión de Datos:
  Los servicios que interactúan con la base de datos serán responsables de manejar las transacciones y consultas. Entre las entidades principales se incluyen:
  - Pedidos (información de productos a enviar).
  - Vuelos (capacidad, frecuencia y rutas).
  - Almacenes (capacidad y ubicación).
  - Corridas/Simulaciones (parámetros y resultados de los algoritmos).

Ejemplo de aplicación:

Pedido.java

src > com > morapack > ga > Pedido.java

```java
package com.morapack.ga;

public class Pedido {
    public String id;
    public String destino;
    public int cantidad;
    public String hubOrigen; // SPIM, EBCI o UBBB

    public Pedido(String id, String destino, int cantidad, String hubOrigen) {
        this.id = id;
        this.destino = destino;
        this.cantidad = cantidad;
        this.hubOrigen = hubOrigen;
    }
}
```

6. API REST

- Versionamiento: todas las rutas estarán bajo /api/v1/**.
- Principios REST: uso de sustantivos en plural, métodos HTTP adecuados (GET, POST, PUT, DELETE), respuestas en JSON y códigos de estado correctos.
- Tecnología: implementación con Spring Boot y Spring Web, utilizando controladores con @RestController.

Ejemplo de aplicacion:

```java
@RestController
@RequestMapping("/api/v1/pedidos")
public class PedidoController {

  private final PedidoService service;

  public PedidoController(PedidoService service) {
    this.service = service;
  }

  @PostMapping
  public ResponseEntity<PedidoDTO> crear(@RequestBody PedidoDTO dto) {
    PedidoDTO creado = service.crear(dto);
    return ResponseEntity.created(URI.create("/api/v1/pedidos/" + creado.id())).body(creado);
  }

  @GetMapping
  public ResponseEntity<List<PedidoDTO>> listar() {
    return ResponseEntity.ok(service.listar());
  }

  @GetMapping("/{id}")
  public ResponseEntity<PedidoDTO> obtener(@PathVariable Long id) {
    return ResponseEntity.ok(service.obtener(id));
  }

  @DeleteMapping("/{id}")
  public ResponseEntity<Void> eliminar(@PathVariable Long id) {
    service.eliminar(id);
    return ResponseEntity.noContent().build();
  }
}
```

7. Autenticación y Autorización

El backend de MoraPack contará con un sistema de autenticación basado en JWT (JSON Web Tokens) para validar la identidad de los usuarios que acceden a los módulos de pedidos, planificador y monitoreo.

Se usará Spring Security para integrar la validación del token en cada petición, restringiendo el acceso a los endpoints según el rol asignado. Por ejemplo:

- ADMIN: gestión completa de escenarios y experimentos.
- PLANNER: ejecución de corridas y replanificaciones.
- VIEWER: solo lectura de métricas y monitoreo.

8. Manejo de Errores

La aplicación definirá excepciones personalizadas (p. ej., BadRequestException para errores de validación, NotFoundException para recursos inexistentes o AlgorithmException para fallas en la ejecución de un metaheurístico).

El tratamiento será centralizado mediante un componente con @ControllerAdvice, lo que permitirá

devolver respuestas en formato JSON claras y consistentes, incluyendo detalles como código de error, mensaje y timestamp

9. Pruebas

El backend incorporará un esquema de pruebas automatizadas para asegurar la calidad del sistema:

- Pruebas unitarias con JUnit 5, enfocadas en validar la lógica de negocio (validación de pedidos, restricciones de tiempo, operadores de los algoritmos).
- Pruebas de integración con Spring Boot Test y MockMvc, orientadas a verificar la comunicación entre controladores, servicios y repositorios, así como la correcta respuesta de los endpoints definidos en la API REST.

10. Despliegue

La aplicación se empacará en un contenedor Docker para simplificar su instalación y ejecución en distintos entornos. Un Dockerfile típico será:

```
FROM openjdk:17-jre-slim
COPY target/morapack-backend.jar app.jar
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

Además, se habilitarán pipelines de Integración y Despliegue Continuo (CI/CD), utilizando herramientas como GitHub Actions, Jenkins o GitLab CI, para automatizar las etapas de compilación, ejecución de pruebas y despliegue de la aplicación en los servidores del laboratorio o en entornos de prueba.

11. Documentación

La solución backend contará con documentación generada automáticamente a partir de los controladores, utilizando Swagger/OpenAPI para exponer los endpoints disponibles. Esta documentación será accesible vía navegador en entornos de desarrollo y permitirá al equipo de MoraPack probar las rutas de la API de manera interactiva.

Adicionalmente, se mantendrá un archivo README.md siempre actualizado, con las instrucciones de instalación, configuración, ejecución y despliegue de la aplicación en los distintos entornos.

12. Seguridad

La aplicación garantizará la seguridad de los datos transmitidos mediante la implementación de **HTTPS** en todos los entornos de ejecución.

Se seguirán las recomendaciones de **OWASP Top 10**, aplicando medidas contra amenazas comunes como inyección de datos, exposición de información sensible o accesos no autorizados. Esto asegura que el planificador y el monitor de operaciones de MoraPack sean confiables y protegidos frente a ataques.

13. Rendimiento

Para mejorar la eficiencia del backend se considerarán dos estrategias principales:

- Caching: uso de Spring Cache o de un sistema de caché externo como Redis, especialmente para acelerar las operaciones de lectura frecuentes (por ejemplo, consultas de métricas históricas de corridas).
- Optimización de consultas: garantizar que las consultas a la base de datos estén correctamente indexadas y optimizadas, evitando sobrecarga en operaciones críticas como la asignación de pedidos y el registro de vuelos.

14. Control de Versiones

El desarrollo del proyecto se gestionará con Git como sistema de control de versiones.
Se seguirá el flujo de trabajo Gitflow, utilizando ramas principales (main, develop) y ramas de soporte (feature/*, fix/*, release/*) para mantener un desarrollo ordenado y trazable.

15. Glosario

- JWT: *JSON Web Token*, estándar para autenticación segura entre cliente y servidor.
- API: *Application Programming Interface*, interfaz que define cómo interactúa el frontend o sistemas externos con el backend.